

Cheap Domains for AWS Lightsail

https://gemini.google.com/app/624b29769cae36cf

User prompt: If i want to cheap domain and cheap domain extension to use it of my engine on AWS lightsail vps ... Not planing to share it with ppl for sure

Response: For an internal quant engine running on an AWS Lightsail VPS with no public audience, your primary goals are **low upfront cost**, **cheap annual renewals** (avoiding renewal price traps), and **free WHOIS privacy**.

Cheapest Recommended TLD Extensions (First-Year & Renewals)

- .xyz (Best Ultra-Cheap Option):** ~\$1.50 to \$2.50 first year; renews at ~\$10 to \$13/year. Widely supported and the standard for disposable or private developer projects.
- .online / .site / .tech:** Often promo-priced at ~\$1.00 to \$3.00 for year one, but watch out: renewals can spike to \$25–\$40/year.
- .dev / .app:** ~\$10 to \$12/year flat (registration and renewal). Enforces HTTPS by default at the browser level, which is great for secure APIs and WebSocket communication.
- .com:** ~\$9.50 to \$10.50/year flat. If you plan to run the engine for multiple years, .com is often cheaper in the long run than promo TLDs that hike renewals.

Best Budget Registrars

Registrar	Pricing Profile	Key Features
Porkbun (<i>Top Pick</i>)	Lowest flat retail pricing across almost all TLDs	Free WHOIS privacy, free SSL/DNSSEC, straightforward UI, no hidden renewal surprises.
Spaceship (by Namecheap)	Ultra-low entry rates (~\$1–\$3 promo TLDs)	Modern dashboard, built-in free privacy, seamless DNS management.
Cloudflare Registrar	Zero-markup wholesale pricing (at-cost)	Cheapest flat renewals (~\$10 for .com), but requires using Cloudflare DNS.

Recommended Setup Strategy for Lightsail

- Register a disposable **.xyz** (or flat **.com**) domain via **Porkbun** or **Spaceship** for ~\$2–\$10.
- Point an **A Record** (e.g., engine.yourdomain.xyz or the apex domain) directly to your Lightsail dedicated Static IPv4 address.
- Use a reverse proxy like **Caddy** or **Nginx + Certbot** on Ubuntu to issue a free automated Let's Encrypt SSL certificate, giving your engine clean HTTPS/WSS endpoints for remote management, dashboards, or secure webhooks.

User prompt: What do u think of OrcaQuant.com .

Response: **OrcaQuant.com** is a strong, punchy domain name. Evaluating it across branding, technical identity, and practical suitability highlights several clear advantages alongside a few considerations:

Why It Works Well

- Institutional Tone with Punch:** "Orca" evokes apex predator imagery (hunting liquidity pools and executing with precision), while "Quant" immediately grounds the platform in systematic, mathematical architecture.
- Phonetics & Memorability:** It rolls off the tongue naturally (two syllables + one syllable), is easy to type without spelling errors, and avoids confusing hyphens or awkward letter combinations.
- Professional .com Authority:** Even for an internal headless deployment or private dashboard, .com remains the gold standard for long-term flat pricing stability without aggressive renewal spikes.

Things to Consider

- Stealth & Discretion:** If this domain is purely for internal WebSockets, API webhooks, and private Lightsail VPS access, a name containing "Quant" is recognizable. If absolute obscurity is the goal, generic names (e.g., app-edge-node, core-sys) blend into standard web traffic more quietly. However, if the server is secured with locked ports, IP whitelisting, and token/key-based access, naming visibility poses no operational security risk.
- Market Associations:** "Orca" is also associated with the Solana DEX (Orca.so) and various commercial trading tools, though combining it into *OrcaQuant* gives it a clean, standalone institutional identity.

Verdict

If available at standard registration rates (\$9–\$11/year flat), **OrcaQuant.com** strikes an excellent balance between sharp branding, memorable typography, and long-term pricing value.

User prompt: Okay lets focus on Stealth & Discretion. What could be matching the vibe and stealth !

Response: To achieve true **stealth, operational obscurity, and an institutional engineering vibe**, we need domain conventions that completely discard retail giveaways like "quant", "trade", "algo", "signals", or "bot".

When automated scanners, network sniffers, or port monitors see these domains, they immediately identify financial microservices. Stealth domains, by contrast, blend seamlessly into standard developer background traffic (telemetry nodes, caching infrastructure, synchronization relays, or generic edge microservices) while still carrying a clean, serious aesthetic.

1. Infrastructure & Telemetry Disguises (Ultimate Stealth) These sound like standard dev-ops or distributed systems nodes, completely masking financial activity:

- `synclogix.net / synclogix.io` — Sounds like a distributed pipeline or database sync layer.
- `nodepulse.dev / nodepulse.xyz` — Sounds like standard server health monitoring.
- `edgerelay.net / edgerelay.io` — Sounds like a simple reverse proxy or CDN edge gateway.
- `streamcore.dev / streamcore.xyz` — Sounds like an internal event stream or WebSocket router.
- `packetflow.net / packetmesh.io` — Sounds like raw network infrastructure.

2. Abstract & Algorithmic Stealth (Minimalist Vibe) Names that retain high-level mathematical and systemic weight without explicitly revealing market execution:

- `nullvector.dev / nullvector.net` — Pure mathematical abstraction.
- `chronoloop.io / chronoloop.dev` — Evokes time-series execution loops cleanly.
- `statevector.xyz / statevector.net` — Sounds like an academic state-machine node.
- `tensornode.dev / tensornode.xyz` — Looks like a generic machine-learning worker.
- `axiomflow.net / axiomsys.io` — Sharp, minimalist, and discreet.

3. Abstract Animal / Code-Name Stealth If you want to keep the "Apex / Stealth Predator" vibe without broadcasting execution:

- `orcanode.net / orcanode.dev` — Replaces "Quant" with "Node", removing the finance flag immediately.
- `krakenrelay.xyz / apexbridge.net` — Code-name aesthetic, blends into private backend architectures.
- `shadowstream.dev / blackvect.net` — Clean, dark-mode terminal vibe.

Stealth Configuration Rules for Your Lightsail Node

- **Default Nginx/Caddy Response:** Set the root domain (`[https://yourdomain.xyz/](https://yourdomain.xyz/)`) to return a blank `200 OK` or a generic `{"status": "healthy"}` JSON payload. Never display a login page or chart canvas on the bare root.
- **Non-Descriptive Subdomains:** Route your execution and MCP APIs through generic subdomains like `api-v1.yourdomain.xyz`, `ws-stream.yourdomain.xyz`, or `telemetry.yourdomain.xyz`.
- **Private Gateways:** Keep the `/api/agent` and `/api/mcp` routes strictly protected behind high-entropy bearer tokens and IP filtering so unauthenticated traffic encounters standard 404 or 403 responses.

Which naming direction fits your setup best: **pure DevOps infrastructure** (e.g., `nodepulse`, `streamcore`), **abstract mathematical** (e.g., `nullvector`, `statevector`), or a **stealth predator code-name** (e.g., `orcanode`)?

User prompt: What if i don't have domain name ? How i can view the dashboard then ?

Response: You do not need to buy or configure a domain name to view, monitor, and interact with your dashboard. When your Next.js application and quant engine run on your AWS Lightsail VPS, you have two primary, standard methods to access your cockpit.

Method 1: Direct VPS Static IP Access (Fastest & Standard)

Your AWS Lightsail instance comes with a dedicated, permanent **Static IPv4 address**. You can access your frontend directly using the raw IP and port in your browser:

- **Direct URL:** `http://YOUR_LIGHTSAIL_STATIC_IP:3000` (or `http://YOUR_STATIC_IP` if mapped to default HTTP port 80 via a reverse proxy like Nginx/Caddy).
- **Network Requirement:** Ensure port 3000 (or 80) is opened in your **AWS Lightsail Networking / Cloud Firewall** tab.
- **Security:** Keep NextAuth/OAuth login active so unauthorized visitors hitting the raw IP cannot access the trading controls.

Method 2: Encrypted SSH Port Forwarding (Maximum Stealth & Security)

If you want absolute privacy—where **no public web ports** are open to the internet and automated bots cannot even scan your IP:

1. Close all web ports (80, 443, 3000) in your Lightsail Firewall, keeping **only your custom SSH port open**.
2. Run an SSH tunnel command from your local terminal:

```
ssh -i /path/to/lightsail_key.pem -L 3000:localhost:3000 ubuntu@YOUR_LIGHTSAIL_STATIC_IP
```

3. Open your browser on your local laptop and visit:

```
http://localhost:3000
```

Your browser treats the remote AWS Lightsail engine as if it were running natively on your local machine, routing all chart updates and WebSocket data through an encrypted SSH tunnel.

Summary Comparison

Approach	URL Format	Open Firewall Ports	Best For
Direct IP Access	http://STATIC_IP:3000	Port 3000 (or 80)	Quick access from mobile phone or laptop with OAuth login protection.
SSH Local Tunnel	http://localhost:3000	Only Port 22 / Custom SSH	100% dark mode / zero public exposure; totally invisible to internet scanners.

User prompt: And if i have domain the advice is using subdomain

Response: **Yes, absolutely.** If you have a custom domain, routing your quant engine and AI bridges through **subdomains** is standard institutional architecture.

Why Using Subdomains is the Superior Approach

- **Operational Decoupling & Isolation:** You can point different subdomains to completely different servers. For example, your Next.js dashboard UI can live on Vercel (app.yourdomain.com), while your headless low-latency execution daemon sits on your AWS Lightsail VPS in Tokyo (engine.yourdomain.com or core.yourdomain.com).
- **Clean Network Routing & SSL:** With subdomains, reverse proxies (like Nginx or Caddy) handle routing, WebSockets, and automated SSL (HTTPS/WSS) termination cleanly without port collisions.
- **Granular Firewall & Security Scoping:** You can put heavy Cloudflare DDoS protection or Google OAuth on your main dashboard subdomain (app.yourdomain.com), while keeping your engine subdomain (mcp.yourdomain.com or api.yourdomain.com) strictly secured with high-entropy M2M bearer tokens and IP whitelisting.

Recommended Subdomain Architecture for Your Setup

Subdomain	Routes To	Purpose	Security Layer
app.yourdomain.com	Next.js Frontend (Vercel / VPS)	Visual trading cockpit, Lightweight Charts, strategy settings UI.	Google OAuth / NextAuth login.
api.yourdomain.com (or core.)	AWS Lightsail VPS (Tokyo)	24/7 Headless quant daemon, WebSocket streams, live execution loop.	Static IP whitelisting & API keys.
mcp.yourdomain.com (or agent.)	Next.js M2M Route (/api/mcp)	Remote MCP tool gateway for Gemini Spark & autonomous agents.	Bearer Secret Header (M2M_AGENT_SECRET).

DNS Configuration on Lightsail VPS (Example)

1. In your registrar (e.g., Porkbun, Cloudflare, Spaceship):
 - Create an **A Record**: core (or engine) → Point to your **AWS Lightsail Static IPv4**.
2. On your Lightsail server:
 - Let Nginx/Caddy listen to core.yourdomain.com and forward traffic locally to port 3000 (or 8000 for Python services).
 - Set your bare apex domain (yourdomain.com) to either return a generic blank 200 OK or redirect away for maximum stealth.

User prompt: Why should i keep vercel if i have vps ? What is the idea ?

Response: **The core reason to keep Vercel while running a VPS comes down to separating stateless UI delivery from stateful, continuous trading operations.**

1. The Division of Labor (Why Both Work Together)

- **Vercel's Role (The Visual Cockpit):** Vercel excels at hosting Next.js frontends on a global Edge Content Delivery Network (CDN). It handles automatic HTTPS, instantaneous asset caching, git-push continuous deployment, and optimized client bundle delivery so you can open your dashboard from a laptop or phone anywhere with zero server configuration.
- **The VPS's Role (The Execution Engine):** A VPS runs a persistent, stateful Linux environment. Unlike Vercel's serverless functions—which spin up on a web request and terminate after 10–60 seconds—a VPS maintains 24/7 background WebSocket connections to Binance, runs continuous order book state machines, and executes trades with sub-millisecond network co-location without relying on browser uptime.

2. Architectural Comparison

Feature / Dimension	Vercel (Frontend & Edge)	AWS Lightsail VPS (Engine)
Execution Model	Stateless Serverless (ephemeral Lambdas)	Stateful, persistent 24/7 background processes
Exchange WebSockets	Cannot maintain 24/7 persistent connections	Keeps permanent TCP socket open to Binance
Static IP for Whitelisting	Dynamic/shared outbound IPs (cannot whitelist)	Dedicated Static IPv4 (required for exchange security)
Deployment Simplicity	100% automated via <code>git push</code>	Requires Linux setup, PM2/systemd, reverse proxy
Asset Delivery & CDN	Global Multi-Region Edge Caching	Single Datacenter (e.g., Tokyo ap-northeast-1)

3. Do You Actually Need to Keep Vercel?

No, keeping Vercel is optional. You can run the entire system in one of two configurations:

- **Hybrid Model (Vercel + VPS):** Keep your Next.js frontend on Vercel for zero-maintenance UI hosting and instant GitHub deployments, while pointing API and trading routes to your AWS Lightsail VPS daemon.
- **All-in-One VPS Model:** Host both the Next.js frontend (using `next start` or Next.js standalone mode under PM2) and the background execution loop entirely on your AWS Lightsail VPS. This eliminates Vercel completely, keeping the whole stack in a single self-contained server.

User prompt: Okay what os the best practice? The real best approach

Response: The institutional best practice for a private quantitative trading terminal is the **Self-Contained All-in-One VPS Architecture** hosted on your AWS Lightsail Tokyo instance.

Relying on a split architecture (Vercel + Serverless Cloud DB + VPS) introduces unnecessary points of failure, cold-start latency, and external quota limits for a system intended solely for your personal use.

The Gold-Standard Architecture Breakdown

- **1. All-in-One Execution & UI on AWS Lightsail (Tokyo ap-northeast-1):** Host your Next.js dashboard UI (running in standalone Node.js mode), your 24/7 background execution daemon, and a local database (PostgreSQL or SQLite) directly on the same Lightsail VPS.
 - *Why this wins:* Zero network hops between UI, database, and execution engine (< 0.2 ms loopback). It completely eliminates third-party database bandwidth caps and serverless runtime timeouts.
 - *Sub-millisecond exchange latency:* Residing on Amazon's Tokyo backbone maintains < 1.5 ms round-trip execution to Binance Futures matching engines.
- **2. Stealth Domain + Cloudflare DNS Proxy (Zero Server Exposure):** Register a cheap, generic stealth domain (e.g., .xyz or .dev via Porkbun/Cloudflare) and route a subdomain like `core.synclogix.net` through **Cloudflare Proxy (Orange Cloud)**.
 - *Why this wins:* Cloudflare masks your VPS Static IP from public scanners, provides free SSL (HTTPS/WSS), and blocks automated bots before they touch your server.
- **3. Caddy Reverse Proxy for Zero-Maintenance HTTPS:** Run **Caddy** on the VPS as your edge web server. Caddy automatically provisions and renews SSL certificates and routes incoming traffic internally to port 3000 (Next.js dashboard) and WebSocket routes without configuring complex SSL scripts.
- **4. Fortified Zero-Trust Security Perimeter:**
 - **Binance Keys:** Whitelist your dedicated Lightsail Static IPv4 and enforce Trade-Only / Read-Only permissions (Withdrawals strictly disabled).
 - **Firewall (UFW / AWS Lightsail):** Allow inbound traffic *only* on Port 443 (HTTPS) and your custom SSH port.
 - **Dashboard Access:** Protect your web cockpit behind Google OAuth (NextAuth) or free Cloudflare Zero Trust Access (one-time pin to your email).

Execution Model Parity

Keep the quantitative engine state machine (3-Stage Harvest: 40% @ 1.0R, 40% @ 1.5R, 20% @ DOL) and trailing stops operating **100% in-memory** within the VPS process. Trade journal updates commit to your local VPS database asynchronously, guaranteeing that live market tick evaluations never stall on disk I/O.

Would you like to review the step-by-step VPS provisioning and Caddy reverse proxy setup plan next, or focus on prepping the Next.js standalone build for PM2 daemonization?

User prompt: Will back to it later, just quick question: i will link my github repo or how i will uploady engine ?

Response: The standard and most reliable method to deploy your engine to your AWS Lightsail VPS is by **cloning your private GitHub repository directly over SSH**.

The Three Ways to Get Your Engine onto the VPS

- **Method 1: Private GitHub Repository via SSH Deploy Key (Recommended Standard)** You push your commits locally from your IDE to your private GitHub repo. On the VPS, you generate a read-only SSH Deploy Key, attach it to your GitHub repository settings, and run a standard git clone.
 - *Why this wins:* Ongoing updates take 5 seconds. Whenever you refine strategy parameters or add features, you simply push locally and run a pull and reload command on the VPS.
- **Method 2: GitHub CLI (gh auth)** You install the official GitHub CLI on your Lightsail instance, authenticate your GitHub account in the terminal, and clone your private repo directly without manually copying SSH public keys.
- **Method 3: Direct Secure Copy (SCP / SFTP / Rsync)** If you prefer not to touch GitHub on the server, you can stream your local project folder directly over your Lightsail SSH connection to the server. While functional, it is clunky for frequent updates because you have to re-upload files manually every time you tweak code.

The Golden Security Separation: Code vs. Secrets

- **What travels through GitHub:** Your Next.js pages, quantitative calculations, state machines, and package dependencies.
- **What stays strictly on the VPS:** Your live .env file (containing your Binance API keys, webhook secrets, and database credentials) is created manually on the VPS and **never committed to GitHub**.

The Ongoing Deployment Loop

1. **Local Machine (IDE):** Develop, test backtests, commit, and push changes to your private branch on GitHub.
2. **Lightsail VPS:** Pull the latest changes, build the production bundle, and let your PM2 process manager reload the background daemon with zero downtime.

User prompt: So i can have 2 branches linked like main and dev for testing on the vps as staging !?

Response: **Yes, absolutely.** Running your main (production) and dev (staging) branches side-by-side on the same AWS Lightsail VPS is a standard and effective deployment pattern.

It gives you a live sandbox where you can test new indicator tweaks, WebSocket optimizations, or AI agent hooks in an identical server environment before merging them into your live trading engine.

How Dual-Branch Hosting Works on a Single VPS

- **1. Independent Directory Clones:** Instead of switching branches inside a single folder, you clone your repository into two distinct directories on the VPS (for example, a production folder tracking main and a staging folder tracking dev).
- **2. Port Isolation:** Each environment runs on its own internal port:
 - **Production (main):** Runs on port 3000.
 - **Staging (dev):** Runs on port 3001 (or 4000).
- **3. Process Supervisor Separation (PM2):** PM2 manages both as independent background daemons (e.g., engine-prod and engine-stage). You can build, restart, crash, or pull updates on dev without causing a single millisecond of downtime or interruption to your live main execution loop.
- **4. Subdomain Reverse Proxy Routing:** Your edge web server (Caddy or Nginx) routes traffic cleanly based on subdomains:
 - core.yourdomain.xyz → forwards internally to localhost:3000 (Live).
 - stage.yourdomain.xyz → forwards internally to localhost:3001 (Staging).
- **5. Strict Environment & Key Isolation (Quant Guardrail):** Each folder contains its own isolated .env configuration file:
 - **Production .env:** Holds your live Binance Futures API keys and production database connection.
 - **Staging .env:** Holds Binance Testnet keys, mock balance states, or sandbox database branches so experimental strategy tests never fire orders against real capital.

Production vs. Staging Architecture Matrix

Environment	Git Branch	Internal Port	Subdomain Routing	API & Capital Scope
Production	main	localhost:3000	core.yourdomain.xyz	Live Binance Keys + Live Database
Staging	dev	localhost:3001	stage.yourdomain.xyz	Testnet / Paper Engine + Test DB

The Deployment Workflow

1. Develop features and backtest locally on your dev branch.
2. Push your dev branch to GitHub.
3. On the VPS, pull updates into your staging folder, build, and verify everything live on `stage.yourdomain.xyz`.
4. Once verified, merge dev into main on GitHub, pull in your VPS production directory, and reload the production process with zero disruption.

User prompt: I can get api key from binance as testing ? It can act like the real api and not effect my real account !?

Response: **Yes, absolutely.** Binance provides a dedicated **Futures Testnet (Sandbox Environment)** that issues separate test API keys and loaded demo funds, ensuring zero risk to your real capital or live account balance.

It functions identically to the live exchange matching engine, using the exact same order types, HMAC-SHA256 signatures, and WebSocket event formats.

Live vs. Testnet Architecture Comparison

Dimension	Live Binance Futures (main)	Binance Futures Testnet (dev / Sandbox)
Capital Risk	Real USDT / USDC balance	100% Virtual / Fake funds (resettable at any time)
REST Base URL	[https://fapi.binance.com] (https://fapi.binance.com)	[https://testnet.binancefuture.com] (https://testnet.binancefuture.com)
WebSocket Stream	wss://fstream.binance.com	wss://fstream.binancefuture.com (or testnet stream pool)
API Key Origin	Generated inside your real Binance account	Generated on the Binance Futures Testnet portal
Account Isolation	Live database & real order books	Completely separate database and mock matching engine

How to Use the Testnet with Your Quant Engine

- **1. Independent Portal Registration:** You sign in to the Binance Futures Testnet portal (using GitHub, Google, or an independent demo login). It generates a dedicated **API Key** and **Secret Key** that only exist on the testnet cluster.
- **2. Free Virtual Margin Faucet:** The testnet dashboard provides a one-click faucet to top up test USDT or USDC whenever needed, allowing you to test order sizing and multi-stage scaling freely.
- **3. Zero-Code Engine Parity:** Because the API schemas, order endpoints (`/fapi/v1/order`), and WebSocket execution reports are identical, transitioning your engine between simulation and live execution requires changing only your `.env` configuration (swapping the Base URL and API credentials).

Strategic Role for Your Staging (dev) Environment

This setup pairs with the dual-branch VPS architecture:

- **Production (main on port 3000):** Ingests live Binance market data and uses live API keys with strict static IP whitelisting and no-withdrawal scopes.
- **Staging (dev on port 3001):** Uses your **Binance Testnet API keys** to route real test orders, verifying that your limit entry cancellations, 3-stage harvest scaling (40/40/20), and orphan order cleanup state machines execute without risking real capital.

User prompt: Okay now review all the conversation and gather all the information into one detailed plan, step by step plan

Response: **Master Deployment & Operational Architecture Plan**

Phase 1: VPS Provisioning & Infrastructure Hardening

- **1.1 Launch Instance:** Deploy an **AWS Lightsail Ubuntu 24.04 LTS** instance in the **Tokyo region (ap-northeast-1)** on the **\$7/month Micro plan** (2 vCPUs, 1 GB RAM, 40 GB SSD, Public IPv4 bundle). This achieves direct network co-location (< 1.5 ms RTT) with Binance Futures matching engines.
- **1.2 Assign Static IPv4:** In the Lightsail Networking tab, create and attach a permanent **Static Public IPv4** address.
- **1.3 SSH & Port Hardening:**
 - Connect via SSH key pair; disable root login and password authentication.
 - In the AWS Lightsail Firewall, lock down all ports except Port 443 (HTTPS), Port 80 (HTTP challenge redirect), and your designated SSH port.

Phase 2: Stealth Domain, DNS & Reverse Proxy Configuration

- **2.1 Domain Registration:** Secure an inexpensive, non-descriptive stealth domain (e.g., .xyz or .dev via Porkbun or Cloudflare) using infrastructure-style naming conventions (e.g., nodepulse, streamcore, synclogix, or orcanode) to blend into standard telemetry traffic without revealing financial execution.
- **2.2 DNS & Cloudflare Proxy:** Route DNS through Cloudflare with proxying enabled (Orange Cloud) to mask your Lightsail Static IP, absorb automated scanners, and manage edge SSL certificates.
- **2.3 Caddy Reverse Proxy Deployment:** Configure Caddy on the VPS to handle automated TLS certificate provisioning and internal port forwarding:
 - Apex domain (yourdomain.xyz) → Returns a blank 200 OK or generic health check payload.
 - Production cockpit (core.yourdomain.xyz) → Forwards internally to localhost:3000.
 - Staging sandbox (stage.yourdomain.xyz) → Forwards internally to localhost:3001.

Phase 3: Repository Setup & Dual-Branch Isolation

- **3.1 SSH Deploy Key:** Generate a read-only SSH key on the Lightsail VPS and link it as a Deploy Key in your private GitHub repository.
- **3.2 Directory Separation on VPS:**
 - /var/www/quant-engine/production → Tracks the main branch.
 - /var/www/quant-engine/staging → Tracks the dev branch.
- **3.3 Isolated Environment Configurations (.env):**
 - production/.env → Live Binance Futures API keys, production database connection, and production Google OAuth/NextAuth credentials.
 - staging/.env → Binance Futures Testnet credentials, sandbox database connection, and staging redirect URIs.
 - *Rule:* Never commit .env files to GitHub; maintain them strictly on the server.

Phase 4: Exchange API Configuration & Sandbox Validation

Environment	Branch	Port	Target Exchange Base URL	API Scope & Restrictions
Production	main	3000	[https://fapi.binance.com] (https://fapi.binance.com)	Live Binance API: Whitelisted VPS Static IPv4, "Enable Reading" + "Enable Futures" ON, "Enable Withdrawals" strictly OFF.
Staging	dev	3001	[https://testnet.binancefuture.com] (https://testnet.binancefuture.com)	Binance Testnet API: Virtual margin sandbox, zero real capital risk, isolated sandbox keys.

- **Execution Order Model:** Resting limit entries execute as **Maker (0.0000% fee)** at structural levels (Mean Threshold / FVG CE). Take Profit tranches are submitted as **Reduce-Only Limit Orders** to eliminate accidental reverse position flips. Stop Losses are submitted as **Stop-Market** orders to guarantee immediate execution on breach.

Phase 5: Database Strategy & Local-First Decoupling

- **5.1 Local VPS Database Setup:** Host PostgreSQL or SQLite locally on the Lightsail VPS to provide unlimited NVMe storage, eliminating cloud database transfer limits, 5 GB egress ceilings, and HTTP 402/53000 quota errors.
- **5.2 Sub-Millisecond Local Loopback:** Local queries execute over 127.0.0.1 in < 0.2 ms, removing external network round-trips.
- **5.3 Non-Blocking Persistence:** In-memory execution state machines (WebSocket ticks, candle analysis, trailing stops) run asynchronously from database writes, ensuring disk I/O or database locks never block sub-second order management.
- **5.4 Local-First Preset Fallback:** Maintain client-side presets in local storage first, with background synchronization to the database, ensuring zero terminal lockouts during network resyncs.

Phase 6: PM2 Process Management, Quant Safeguards & MCP Bridge

- **6.1 PM2 Daemonization:**
 - Configure PM2 to supervise engine-prod (port 3000) and engine-stage (port 3001) with automatic restarts on crash and persistent server reboot hooks (pm2 startup && pm2 save).
- **6.2 Production Quant Guardrails:**
 - **3-Stage Harvest Engine:** 40% @ 1.0R (Stop Loss trails to FVG CE / Breakeven), 40% @ 1.5R (Stop Loss ratchets to +1.0R profit floor), 20% @ macro Draw on Liquidity (DOL).
 - **Anti-Micro-Friction Clamp:** Minimum 0.15% Stop Loss distance buffer to protect micro-wicks against fee slippage and spread spikes.

- **Single-Position Concurrency Lock:** Enforce `maxOpenPositions: 1` with a 60-second post-trade cooldown and single-use zone consumption.
- **Orphan Order Purge:** Automated batch cancellation routine executes instantly when position size reaches 0.0 to wipe unhit Take Profit limits.

- **6.3 Remote MCP Server Bridge (/api/mcp):**

- Expose the stateless remote MCP endpoint for Gemini Spark via HTTPS, secured with high-entropy Bearer tokens (`M2M_AGENT_SECRET`) and proxy bypass rules.
- Expose `get_market_context` (market structure, FVGs, SMT) and `submit_quant_decision` (with pre-flight price invalidation gates).

Phase 7: End-to-End Launch & Verification Sequence

- **Step 1 (Staging Test):** Deploy the dev branch to the staging directory on port 3001. Connect Binance Testnet API keys and trigger mock test orders to verify that 3-stage scaling, trailing stop ratchets, and orphan order purges operate cleanly.
 - **Step 2 (OAuth & Proxy Validation):** Verify Google OAuth login redirection on `core.yourdomain.xyz` while confirming that `/api/mcp` and `/api/agent` bypass authentication redirects.
 - **Step 3 (Live Promotion):** Merge verified changes into `main`, pull updates into the production VPS directory, build the standalone Next.js bundle, and reload `engine-prod` in PM2 with zero downtime.
 - **Step 4 (Live Arming):** Arm the production engine with live Binance API keys bound to your dedicated Lightsail Static IP, monitoring live execution telemetry directly on the dashboard.
-